

March 26, 2026

```
[131]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, getcontext # Besonders für sig.
↳ Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
```

```

import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

```

```

[132]: currentversuch = "212"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\212

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\212\Diagramme

Runden signifikanter Stellen

```

[133]: def round_sig_digs(val, errVal):
        """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↪ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert

        **errVal** : float
            Ungenauigkeit des Messwertes

```

```

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
    """

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
    exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_CEILING) *
↳scale # mit scale wird das Komma so verschoben, dass alle
↳signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳* scale

res = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), res

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10^e Prefixe beinhalten)

```

Gaussische Fehlerfortpflanzung

```

[134]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----

```

```

**func** : sympy function
    Funktion dessen Fehler bestimmt werden soll.

**errPronePar** : Array
    Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
    Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ ungleich den Werten für x, y, z.
    Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪ genutzt und für deren Fehler err_x, err_y, err_z etc.

Return
-----

**absolut_err** : sympy function
    Gibt die Fehlergleichung des absoluten Fehlers wieder.

**relativ_err** : sympy function
    Gibt die Fehlergleichung des relativen Fehlers wieder.

**errProneParameters** : array
    Liste aller Fehlerbehafteten Größen
    """

error = 0
errProneParameters = []

function = sp.sympify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion ↪
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
    term = sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden ↪
    ↪ quadratisch aufsummiert
    errProneParameters.append((variable,Delta))

absolute_err = sp.simplify(sp.sqrt(error), rational = True)
relative_err = sp.simplify(sp.sqrt(error/function**2), rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function, long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")

```

```

display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
print(sp.latex(absolute_err))
print("Relativer Fehler:")
display(Math(sp.latex(relative_err,long_frac_ratio=2)))
print(sp.latex(relative_err))
return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[135]: def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
        """
        Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
        ↪Messwert und einem Literaturwert.
        """
        if err_p1 == 0 and err_p2 == 0:
            raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
            ↪fehlerbehaftet sein!")
        else:
            abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

            exp = int(math.floor(math.log10(float(abweichung))))
            if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
            ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
            ↪Nachkommastellenposition hinzu
                exp -= 1
            scale = Decimal(10) ** exp
            abweichung_round = (abweichung / scale).quantize(Decimal('1'),
            ↪rounding=ROUND_CEILING) * scale
            res = f"\num{{{abweichung_round}}}"

            return float(abweichung_round),res

```

```

[136]: geschwindigkeiten=np.array([0.0301,0.0253,0.0201,0.0148,0.0115,0.0071,0.0044,0.
    ↪00192,0.00060])
delta_geschwindigkeiten=np.array([0.0007,0.0007,0.0005,0.0006,0.0009,0.0008,0.
    ↪0005,0.00020,0.00016])
radien=np.array([9,8,7,6,5,4,3,2,1])*1e-3/2
delta_r=0.025*1e-3/2
R=75*1e-3/2

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))

ax.set_title('')
ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
ax.set_xlabel(r'Kugelradien  $r^2$  [ $\mathrm{m}^2$ '])
# ax.set_ylim(())

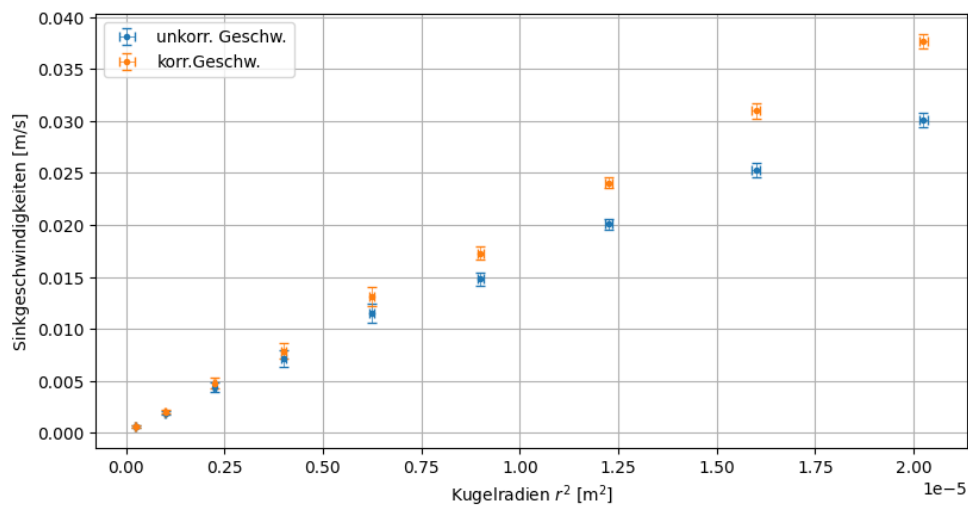
```

```

# ax.set_xlim(())
ax.
    ↳ errorbar(x=radien**2,y=geschwindigkeiten,xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten
    ↳ ",capsize=3,elinewidth=0.5,label='unkorr. Geschw. ')
ax.errorbar(x=radien**2,y=geschwindigkeiten*(1+2.1*radien/
    ↳ R),xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten,fmt="
    ↳ ",capsize=3,elinewidth=0.5,label='korr.Geschw. ')

ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Geschwindigkeiten.png",format="png")

```



```

[137]: # komplett useless berechnung, der Fehler ist so klein
Berechnung=gff("v*(1+2.1*r/R)", "v,r",False)[0]
Fehler_Berechnung=gff("v*(1+2.1*r/R)", "v,r",False)[1]
ergebnisse=[]
delta_ergebnisse=[]

for i in range(len(radien)):
    v=geschwindigkeiten[i]
    Delta_v=delta_geschwindigkeiten[i]
    r=radien[i]
    Delta_r=delta_r
    delta_ergebnisse.append(eval(str(Fehler_Berechnung)))
    ergebnisse.
    ↳ append(round_sig_digs(eval(str(Berechnung)),delta_ergebnisse[i])[0])
    delta_ergebnisse[i]=round_sig_digs(ergebnisse[i],delta_ergebnisse[i])[1]

```

```

ergebnisse=np.array(ergebnisse)
delta_ergebniss=np.array(delta_ergebnisse)
print(ergebnisse)
print(delta_ergebnisse)
print(delta_geschwindigkeiten)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
x = np.linspace(0,2.1*1e-5,100)
ax.set_title('')
ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
ax.set_xlabel(r'Kugelradien  $r^2$  [ $\mathrm{m}^2$ ']')
# ax.set_ylim(())
# ax.set_xlim(())
ax.
    ↳errorbar(x=radien**2,y=geschwindigkeiten,xerr=2*radien*delta_r,yerr=delta_geschwindigkeiten
    ↳",capsize=3,elinewidth=0.5,label='unkorr. Geschw.>')
ax.
    ↳errorbar(x=radien**2,y=ergebnisse,xerr=2*radien*delta_r,yerr=delta_ergebnisse,fmt="
    ↳",capsize=3,elinewidth=0.5,label='korr.Geschw.>')

def lin(x,a):
    return (a*x)
popt, pcov = curve_fit(lin, radien**2,ergebnisse, sigma =_
    ↳delta_ergebnisse,absolute_sigma = True)
ax.errorbar(x, lin(x,*popt), label = "fit", color = "green")

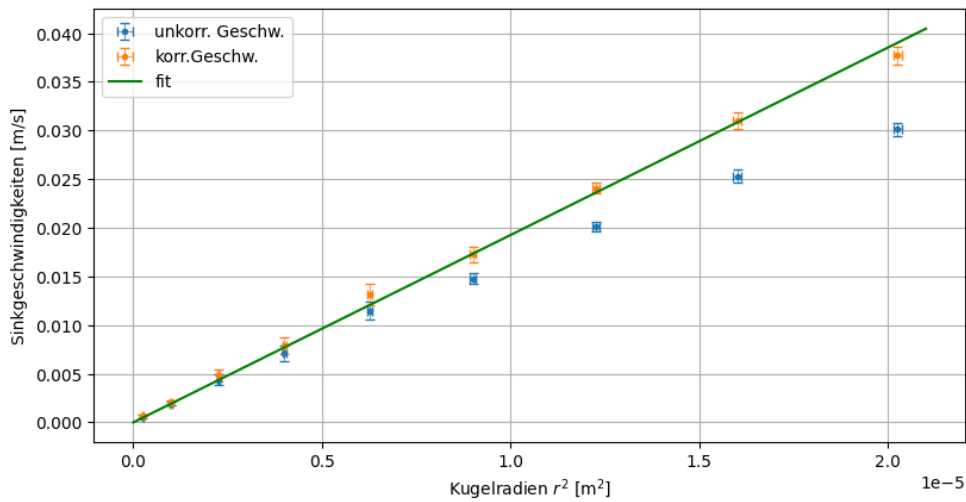
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Fit.png",format="png")
print(popt[0],np.sqrt(pcov[0][0]))

```

```

[0.0377  0.031   0.0241  0.0173  0.0132  0.0079  0.0048  0.00203 0.00062]
[0.0009, 0.0009, 0.0006, 0.0008, 0.0011, 0.0009, 0.0006, 0.00022, 0.00017]
[0.0007  0.0007  0.0005  0.0006  0.0009  0.0008  0.0005  0.0002  0.00016]

```



1926.7486784066791 26.219508117516206

```
[138]: rho_K = 1.385*1e-3/1e-6
Delta_rho_K = 0.0015*1e-3/1e-6
rho_F = 1.1478*1e-3/1e-6
Delta_rho_F = 0.0005*1e-3/1e-6
L_kap = 100e-3
Delta_L_kap = 0.5e-3

g=9.80984
Delta_g=0.00002
a=popt[0]
Delta_a=np.sqrt(pcov[0][0])

eta = eval(str(gff("2 * (rho_K - rho_F) * g / (9 * _
    ↪a)", "rho_K,rho_F,a,g",True)[0]))
Delta_eta = eval(str(gff("2 * (rho_K - rho_F) * g / (9 * _
    ↪a)", "rho_K,rho_F,a,g",False)[1]))
print(round_sig_digs(eta,Delta_eta)[2])
```

Funktion:

$$\frac{g}{9a} (-2\rho_F + 2\rho_K)$$

$$\frac{g}{9a} (-2\rho_F + 2\rho_K)$$

Absoluter Fehler:

$$\frac{2}{9} \sqrt{\frac{1}{a^4} \left(\Delta_a^2 g^2 (\rho_F - \rho_K)^2 + a^2 \left(\Delta_g^2 (-\rho_F + \rho_K)^2 + \Delta_{\rho_F}^2 g^2 + \Delta_{\rho_K}^2 g^2 \right) \right)}$$

$$\frac{\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2(-\rho_F + \rho_K)^2}{g^2(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_K}^2}{(\rho_F - \rho_K)^2}}}{\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2(-\rho_F + \rho_K)^2}{g^2(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_K}^2}{(\rho_F - \rho_K)^2}}} \cdot \frac{g^2 \left(\rho_F - \rho_K \right)^2 + a^2 \left(\Delta_g^2 \left(-\rho_F + \rho_K \right)^2 + \Delta_{\rho_F}^2 g^2 + \Delta_{\rho_K}^2 g^2 \right)}{a^4}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2(-\rho_F + \rho_K)^2}{g^2(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho_K}^2}{(\rho_F - \rho_K)^2}} \cdot \frac{g^2 \left(\rho_F - \rho_K \right)^2 + a^2 \left(\Delta_g^2 \left(-\rho_F + \rho_K \right)^2 + \Delta_{\rho_F}^2 g^2 + \Delta_{\rho_K}^2 g^2 \right)}{a^4} \cdot 0.269(0.005)$$

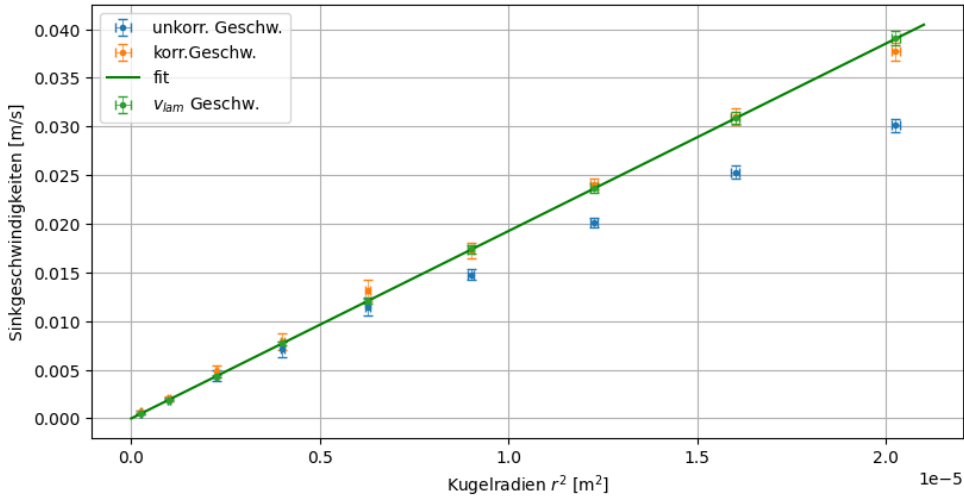
```
[139]: v_lam = 2 * g * (rho_K - rho_F) * radien**2 / (9*eta)
Delta_v_lam = v_lam * np.sqrt((Delta_g/g)**2 + (2 * Delta_r/radien)**2 +
    (Delta_eta/eta)**2 + (Delta_rho_F/(rho_K - rho_F))**2 + (Delta_rho_K/(rho_K -
    rho_F))**2)
for i in range(len(v_lam)):
    v_lam[i] = round_sig_digs(v_lam[i], Delta_v_lam[i])[0]
    Delta_v_lam[i] = round_sig_digs(v_lam[i], Delta_v_lam[i])[1]
    print(f"\num{{{radien[i]*1e3}}}&\num{{{ergebnisse[i]}(delta_ergebniss[i])}}&{round_sig_
print(v_lam, Delta_v_lam)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
x = np.linspace(0, 2.1*1e-5, 100)
ax.set_title('')
ax.set_ylabel('Sinkgeschwindigkeiten [m/s]')
ax.set_xlabel(r'Kugelradien $r^2$ [$\mathrm{m}^2$]')
# ax.set_ylim(())
# ax.set_xlim(())
ax.
    errorbar(x=radien**2, y=geschwindigkeiten, xerr=2*radien*delta_r, yerr=delta_geschwindigkeiten
    ", capsize=3, elinewidth=0.5, label='unkorr. Geschw.')
ax.
    errorbar(x=radien**2, y=ergebnisse, xerr=2*radien*delta_r, yerr=delta_ergebnisse, fmt="
    ", capsize=3, elinewidth=0.5, label='korr. Geschw.')

def lin(x, a):
    return (a*x)
popt, pcov = curve_fit(lin, radien**2, ergebnisse, sigma =
    delta_ergebnisse, absolute_sigma = True)
ax.errorbar(x, lin(x, *popt), label = "fit", color = "green")
ax.errorbar(x=radien**2, y=v_lam, xerr=2*radien*delta_r, yerr=Delta_v_lam, fmt="
    ", capsize=3, elinewidth=0.5, label='$v_{lam}$ Geschw.')
```

```
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Fitmitlam.png",format="png")
```

```
\num{4.500000000000001}&\num{0.0377(0.0009)}&\num{0.0391(0.0007)}\\
\num{4.0}&\num{0.031(0.0009)}&\num{0.0309(0.0006)}\\
\num{3.5}&\num{0.0241(0.0006)}&\num{0.0237(0.0005)}\\
\num{3.0}&\num{0.0173(0.0008)}&\num{0.0174(0.0004)}\\
\num{2.5}&\num{0.0132(0.0011)}&\num{0.01205(0.00024)}\\
\num{2.0}&\num{0.0079(0.0009)}&\num{0.00771(0.00016)}\\
\num{1.5}&\num{0.0048(0.0006)}&\num{0.00434(0.00011)}\\
\num{1.0}&\num{0.00203(0.00022)}&\num{0.00193(0.00006)}\\
\num{0.5}&\num{0.00062(0.00017)}&\num{0.00049(0.00003)}\\
[0.0391 0.0309 0.0237 0.0174 0.01205 0.00771 0.00434 0.00193 0.00049]
[7.0e-04 6.0e-04 5.0e-04 4.0e-04 2.4e-04 1.6e-04 1.1e-04 6.0e-05 3.0e-05]
```



```
[140]: gff("2 * g * (rho_K - rho_F) * r**2 / (9*eta)", "g,rho_K,rho_F,r,eta", True)
```

Funktion:

$$\frac{2gr^2}{9\eta} (-\rho_F + \rho_K)$$

$$\frac{2}{9} \sqrt{\frac{r^2}{\eta^4}} \left(\Delta_{\eta}^2 g^2 r^2 (\rho_F - \rho_K)^2 + \eta^2 \left(\Delta_g^2 r^2 (-\rho_F + \rho_K)^2 + 4\Delta_r^2 g^2 (-\rho_F + \rho_K)^2 + \Delta_{\rho_F}^2 g^2 r^2 + \Delta_{\rho_K}^2 g^2 r^2 \right) \right)$$

Absoluter Fehler:

$$\frac{2}{9} \sqrt{\frac{r^2}{\eta^4}} \left(\Delta_{\eta}^2 g^2 r^2 (\rho_F - \rho_K)^2 + \eta^2 \left(\Delta_g^2 r^2 (-\rho_F + \rho_K)^2 + 4\Delta_r^2 g^2 (-\rho_F + \rho_K)^2 + \Delta_{\rho_F}^2 g^2 r^2 + \Delta_{\rho_K}^2 g^2 r^2 \right) \right)$$

$$\frac{\sqrt{\frac{r^2}{\Delta_{\eta}^2} \left(\Delta_{\eta}^2 r^2 \left(\rho_F - \rho_K \right)^2 + \eta^2 \left(\Delta_g^2 r^2 \left(-\rho_F + \rho_K \right)^2 + 4 \Delta_r^2 g^2 \left(-\rho_F + \rho_K \right)^2 + \Delta_{\rho F}^2 r^2 + \Delta_{\rho K}^2 r^2 \right) \right)}}{\eta^2 + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{4 \Delta_r^2 (-\rho_F + \rho_K)^2}{r^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{\eta}^2}{\eta^2} + \frac{\Delta_g^2 (-\rho_F + \rho_K)^2}{g^2 (\rho_F - \rho_K)^2} + \frac{4 \Delta_r^2 (-\rho_F + \rho_K)^2}{r^2 (\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2}}$$

$$\sqrt{\frac{\Delta_{\eta}^2}{\eta^2} + \frac{\Delta_g^2}{g^2} \left(\frac{(-\rho_F + \rho_K)^2}{(\rho_F - \rho_K)^2} + \frac{4 \Delta_r^2}{r^2} \left(\frac{(-\rho_F + \rho_K)^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho F}^2}{(\rho_F - \rho_K)^2} + \frac{\Delta_{\rho K}^2}{(\rho_F - \rho_K)^2} \right) \right)}$$

```
[140]: (2*g*r**2*(-rho_F + rho_K)/(9*eta),
        2*sqrt(r**2*(Delta_eta**2*g**2*r**2*(rho_F - rho_K)**2 +
eta**2*(Delta_g**2*r**2*(-rho_F + rho_K)**2 + 4*Delta_r**2*g**2*(-rho_F +
rho_K)**2 + Delta_rho_F**2*g**2*r**2 + Delta_rho_K**2*g**2*r**2))/eta**4)/9,
        sqrt(Delta_eta**2/eta**2 + Delta_g**2*(-rho_F + rho_K)**2/(g**2*(rho_F -
rho_K)**2) + 4*Delta_r**2*(-rho_F + rho_K)**2/(r**2*(rho_F - rho_K)**2) +
Delta_rho_F**2/(rho_F - rho_K)**2 + Delta_rho_K**2/(rho_F - rho_K)**2),
        [(g, Delta_g),
         (rho_K, Delta_rho_K),
         (rho_F, Delta_rho_F),
         (r, Delta_r),
         (eta, Delta_eta)])
```

```
[141]: Re = rho_F * geschwindigkeiten * 2*radien / eta
Delta_Re = Re*np.sqrt((Delta_r/radien)**2 + (Delta_eta/eta)**2 + (Delta_rho_F/
rho_F)**2 + (delta_geschwindigkeiten/geschwindigkeiten)**2)
for i in range(len(Re)):
    Re[i]=round_sig_digs(Re[i],Delta_Re[i])[0]
    Delta_Re[i]=round_sig_digs(Re[i],Delta_Re[i])[1]
print(Re,Delta_Re)
```

```
[1.16  0.87  0.602  0.38  0.246  0.122  0.057  0.0165  0.0026] [0.04  0.03
0.018  0.017  0.02  0.014  0.007  0.0018  0.0007]
```

```
[142]: gff("rho_F * v * 2*r / eta","rho_F,v,r,eta",True)
```

Funktion:

$$\frac{2r}{\eta} \rho_F v$$

$$\frac{2 r \rho_F v}{\eta}$$

Absoluter Fehler:

$$2\sqrt{\frac{1}{\eta^4} \left(\Delta_\eta^2 r^2 \rho_F^2 v^2 + \eta^2 \left(\Delta_r^2 \rho_F^2 v^2 + \Delta_{\rho_F}^2 r^2 v^2 + \Delta_v^2 r^2 \rho_F^2 \right) \right)}$$

$$2 \sqrt{\frac{\Delta_\eta^2 r^2 \rho_F^2 v^2 + \eta^2 \left(\Delta_r^2 \rho_F^2 v^2 + \Delta_{\rho_F}^2 r^2 v^2 + \Delta_v^2 r^2 \rho_F^2 \right)}{\eta^4}}$$

Relativer Fehler:

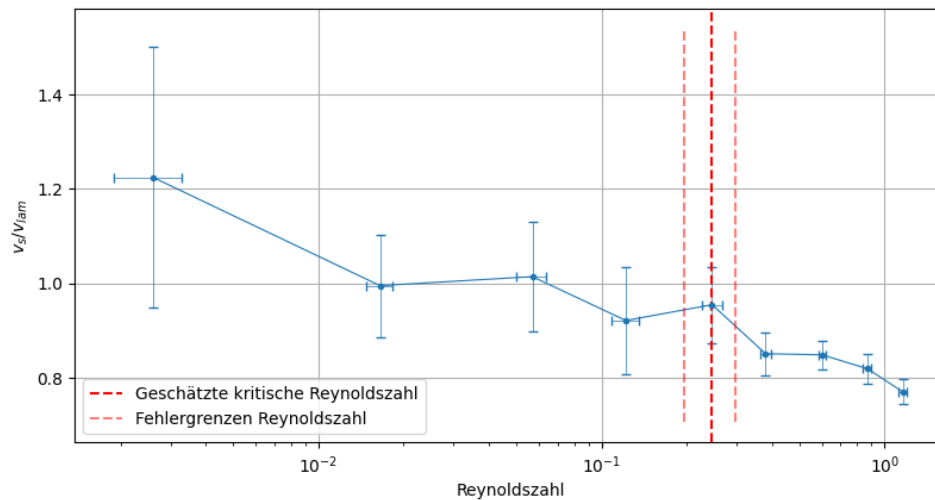
$$\sqrt{\frac{\Delta_\eta^2}{\eta^2} + \frac{\Delta_r^2}{r^2} + \frac{\Delta_{\rho_F}^2}{\rho_F^2} + \frac{\Delta_v^2}{v^2}}$$

$$\sqrt{\frac{\Delta_\eta^2 r^2 \rho_F^2 v^2}{\eta^4} + \frac{\Delta_r^2 \rho_F^2 v^2}{r^2} + \frac{\Delta_{\rho_F}^2 r^2 v^2}{\rho_F^2} + \frac{\Delta_v^2 r^2 \rho_F^2}{v^2}}$$

```
[142]: (2*r*rho_F*v/eta,
        2*sqrt((Delta_eta**2*r**2*rho_F**2*v**2 + eta**2*(Delta_r**2*rho_F**2*v**2 +
        Delta_rho_F**2*r**2*v**2 + Delta_v**2*r**2*rho_F**2))/eta**4),
        sqrt(Delta_eta**2/eta**2 + Delta_r**2/r**2 + Delta_rho_F**2/rho_F**2 +
        Delta_v**2/v**2),
        [(rho_F, Delta_rho_F), (v, Delta_v), (r, Delta_r), (eta, Delta_eta)])
```

```
[143]: vs_vlam = geschwindigkeiten/v_lam
Delta_vs_vlam = np.sqrt((delta_geschwindigkeiten/geschwindigkeiten)**2 +
    ↪(geschwindigkeiten * Delta_v_lam/v_lam**2)**2)

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_xscale('log')
ax.set_ylabel('$v_s/v_{\text{lam}}$')
ax.set_xlabel('Reynoldszahl')
# ax.set_ylim(())
# ax.set_xlim(())
ax.errorbar(x=Re,y=vs_vlam,xerr=Delta_Re,yerr=Delta_vs_vlam,fmt=".",
    ↪",capsize=3,elinewidth=0.5)
ax.plot(Re,vs_vlam,linestyle="--",color="C0",linewidth=0.8)
ax.axvline(0.246, color = "red", linestyle = "dashed", label = "Geschätzte
    ↪kritische Reynoldszahl")
ymin,ymax=ax.get_ylim()
ax.vlines(x=[0.196,0.296],ymin=ymin,ymax=ymax,color = "red", linestyle =
    ↪"dashed", alpha=0.5, label = "Fehlergrenzen Reynoldszahl")
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Reynoldszahl.png",format="png")
```



```
[144]: #Aufgabe 2 Daten
r_kap = 0.75e-3
Delta_r_kap = 0.005e-3
L_kap = 100e-3
Delta_L_kap = 0.5e-3
rho_F=1.1474*1e-3/1e-6
Delta_rho_F=0.0006*1e-3/1e-6
V = np.linspace(10,30,5)* 1e-6
Delta_V=1e-6
T = np.array([344.2,514.4,689.8,862.5,1033.8])
Delta_T = 1
h = 540*1e-3
Delta_h = 0.7*1e-3

p = rho_F * g * h
Delta_p = p*np.sqrt((Delta_h/h)**2 + (Delta_rho_F/rho_F)**2 + (Delta_g/g)**2)
print(round_sig_digs(p,Delta_p)[0],round_sig_digs(p,Delta_p)[1])
print(p,Delta_p)
dVdt=np.mean(V/T)
Delta_dVdt=np.sqrt((np.std(V/T))**2)
# +np.mean((V/T)*np.sqrt(Delta_T**2/T**2 + Delta_V**2/
↪ V**2))**2)
print(round_sig_digs(dVdt,Delta_dVdt)[0],round_sig_digs(dVdt,Delta_dVdt)[1])
eta2 = np.pi * p * r_kap**4 / (8*dVdt*L_kap)
Delta_eta2 = eta2 * np.sqrt((Delta_p/p)**2 + (4*Delta_r_kap/r_kap)**2 +
↪ (Delta_dVdt/dVdt)**2 + (Delta_L_kap/L_kap)**2)
print(round_sig_digs(eta2,Delta_eta2)[0],round_sig_digs(eta2,Delta_eta2)[1])
```

6079.0 9.0

```
6078.1376246400005 8.495999425292306
2.905e-08 7e-11
0.261 0.008
```

```
[145]: def linm(x, m, b):
        return m * x + b
popt, pcov = curve_fit(linm, T, V)
print(round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[0], round_sig_digs(popt[0],
    ↳ np.sqrt(pcov[0][0]))[1])
a=round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[0]
delta_a=round_sig_digs(popt[0], np.sqrt(pcov[0][0]))[1]

eta2 = np.pi * p * r_kap**4 / (8*dVdt*L_kap)
Delta_eta2 = eta2 * np.sqrt(((Delta_p/p)**2 + (4*Delta_r_kap/r_kap)**2 +
    ↳ (delta_a/a)**2 + (Delta_L_kap/L_kap)**2))
print(round_sig_digs(eta2, Delta_eta2)[0], round_sig_digs(eta2, Delta_eta2)[1])
```

```
2.895e-08 8e-11
0.261 0.008
```

```
[146]: Re2 = 2 * rho_F/eta2 * dVdt/(np.pi * r_kap)
Delta_Re2 = Re2 * np.sqrt(((Delta_dVdt)/dVdt)**2 + (Delta_rho_F/rho_F)**2 +
    ↳ (Delta_eta2/eta2)**2 + (Delta_r_kap/r_kap)**2)
print(round_sig_digs(Re2, Delta_Re2)[2])
```

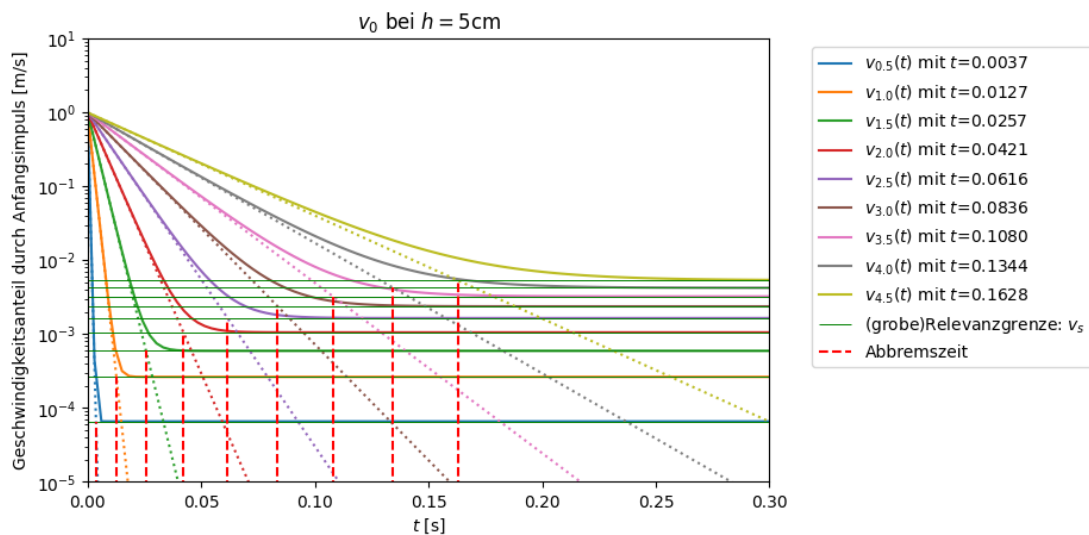
```
\num{0.109(0.004)}
```

ZUSATZ WENIGSTENS NOCHMAL NE INTERESSANTE SACHE

```
[147]: plt.close()
plt.figure(figsize=figsize_cm(12,6))
t = np.linspace(0,0.3,100) #Wir schauen uns Zeiten von 0 bis 1s in 100er
    ↳ Schritten an und plotten das.
r = np.array([1,2,3,4,5,6,7,8,9])*1e-3/2
a = 6 * c.pi * 0.2 * r/(4/3*c.pi*1.385e3*r**3)
v_s = 2/9*(1.385-1.148)*1e3*r**2/0.2
v_0=np.sqrt(2*9.81*0.05)
t_schnitt = np.log(v_s / v_0) / -a
def velocity(t,a,v,v_s):
    return v_s*(1-exp(-a*t))+v * np.exp(-a*t)
def velocity2(t,a,v):
    return v*np.exp(-a*t)
for i in range(len(r)):
    plt.plot(t,velocity(t,a[i],v_0,v_s[i]), label = f"$v_{\{r[i]*1e3:.1f\}}(t)$",
    ↳ mit $t$={t_schnitt[i]:.4f})
    plt.plot(t,velocity2(t,a[i],v_0), color=f"C{i}",linestyle = "dotted")

plt.hlines(y = v_s, xmin = 0, xmax = 2, color = 'green', linewidth=0.7,label =
    ↳ '(grobe)Relevanzgrenze: $v_s$')
```

```
plt.vlines(x=t_schnitt,ymin=1e-20,ymax=v_s, color='red', linestyle='dashed',
          label=f'Abbremszeit')
plt.yscale('log')
plt.ylim((1e-5,1e1))
plt.xlim((0,0.3))
plt.xlabel('$t$ [s]')
plt.ylabel('Geschwindigkeitsanteil durch Anfangsimpuls [m/s]')
plt.title('$v_0$ bei $h=5$cm')
plt.legend(bbox_to_anchor=(1.05, 1),loc='upper left')
plt.tight_layout()
plt.show()
plt.savefig(Ausgabe_path/"Zusatzreal.png",format="png")
```



[148]: `gff("V/T", "V,T", True)`

Funktion:

$$\frac{V}{T}$$

$$\frac{V}{T}$$

Absoluter Fehler:

$$\sqrt{\frac{1}{T^4} (\Delta_T^2 V^2 + \Delta_V^2 T^2)}$$

$$\sqrt{\frac{\Delta_T^2 V^2 + \Delta_V^2 T^2}{T^4}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_V^2}{V^2}}$$

$$\sqrt{\frac{\Delta_{\{T\}}^2}{T^2}} + \frac{\Delta_{\{V\}}^2}{V^2}$$

```
[148]: (V/T,
        sqrt((Delta_T**2*V**2 + Delta_V**2*T**2)/T**4),
        sqrt(Delta_T**2/T**2 + Delta_V**2/V**2),
        [(V, Delta_V), (T, Delta_T)])
```

```
[149]: gff("pi * p * r_kap**4 /(8*dVdt*L_kap)", "p,r_kap,dVdt,L_kap", True)
```

Funktion:

$$\frac{\pi p r_{kap}^4}{8 L_{kap} dV dt}$$

$$\frac{\pi p r_{\{kap\}}^4}{8 L_{\{kap\}} dV dt}$$

Absoluter Fehler:

$$\frac{\pi}{8} \sqrt{\frac{r_{kap}^6}{L_{kap}^4 dV dt^4} \left(\Delta_{L_{kap}}^2 dV dt^2 p^2 r_{kap}^2 + \Delta_{dV dt}^2 L_{kap}^2 p^2 r_{kap}^2 + L_{kap}^2 dV dt^2 \left(\Delta_p^2 r_{kap}^2 + 16 \Delta_{r_{kap}}^2 p^2 \right) \right)}$$

$$\frac{\pi \sqrt{\frac{r_{\{kap\}}^6}{L_{\{kap\}}^4 dV dt^4} \left(\Delta_{L_{\{kap\}}}^2 dV dt^2 p^2 r_{\{kap\}}^2 + \Delta_{dV dt}^2 L_{\{kap\}}^2 p^2 r_{\{kap\}}^2 + L_{\{kap\}}^2 dV dt^2 \left(\Delta_p^2 r_{\{kap\}}^2 + 16 \Delta_{r_{\{kap\}}}^2 p^2 \right) \right)}}{L_{\{kap\}}^4 dV dt^4}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{L_{kap}}^2}{L_{kap}^2} + \frac{\Delta_{dV dt}^2}{dV dt^2} + \frac{\Delta_p^2}{p^2} + \frac{16}{r_{kap}^2} \Delta_{r_{kap}}^2}$$

$$\sqrt{\frac{\Delta_{L_{\{kap\}}}^2}{L_{\{kap\}}^2} + \frac{\Delta_{dV dt}^2}{dV dt^2} + \frac{\Delta_p^2}{p^2} + \frac{16}{r_{\{kap\}}^2} \Delta_{r_{\{kap\}}}^2}$$

```
[149]: (pi*p*r_kap**4/(8*L_kap*dVdt),
        pi*sqrt(r_kap**6*(Delta_L_kap**2*dVdt**2*p**2*r_kap**2 +
        Delta_dVdt**2*L_kap**2*p**2*r_kap**2 + L_kap**2*dVdt**2*(Delta_p**2*r_kap**2 +
        16*Delta_r_kap**2*p**2))/(L_kap**4*dVdt**4))/8,
        sqrt(Delta_L_kap**2/L_kap**2 + Delta_dVdt**2/dVdt**2 + Delta_p**2/p**2 +
        16*Delta_r_kap**2/r_kap**2),
        [(p, Delta_p),
         (r_kap, Delta_r_kap),
         (dVdt, Delta_dVdt),
         (L_kap, Delta_L_kap)])
```

```
[150]: print(sigma_abweichung(0.268,0.261,0.018,0.004)[1])
        print(sigma_abweichung(0.268,0.261,0.008,0.004)[1])
```

\num{0.4}

\num{0.8}